

Bài 9 Thư viện lập trình Agent

1. Software Agent
2. Middleware

(c) SE/FIT/HUT 2002

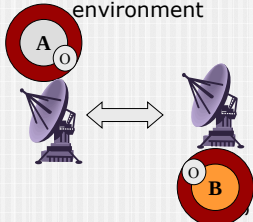
Middleware

- Distributed computing
 - Linking large numbers of different components to perform joint tasks
- Providing support services
 - To enable reuse of code
 - To abstract away from implementation level details
 - To provide uniform access to functions
- Middleware defines (part of...) the environment.
 - Services available: communication, security, etc.
 - Agents live in a software environment – middleware plays a large part

(c) SE/FIT/HUT 2002

Fat Client – Thin Middleware

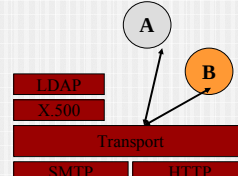
- Agents have many general purpose components
 - Little is left to the environment
- Advantages
 - More control over behaviour
 - Good sense of the environment
 - Agent is more "independent"
- Disadvantages
 - Little abstraction
 - Little interoperability (many different implementations)
 - (Potentially) Less efficient due to multiple specialisation.



SE/FIT/HUT 2002

Thin Client Fat Middleware

- Agents are only behaviour code
 - Everything else is a "service" in the middleware
- Advantages
 - Reduction in code that needs to be written
 - Greater code reuse
 - Stronger Interoperability (more predictable environment)
 - Lightweight agents
- Disadvantages
 - Less control of behaviour
 - No low level environment sensors
 - (Potentially) Less efficient due to generalisation



(c) SE/FIT/HUT 2002

Agent Middleware

- Often quite "fat"
 - Using agents as an abstraction
 - Focus on high level behavioral details
 - Many systems are prototypes so:
 - Efficiency is less of a concern
 - The domain is restricted (less worried about general survival)
- Often Incorporate
 - Naming services
 - Message Transport
 - Communication mechanisms and sometimes even coordination mechanisms such as auction interfaces
 - Agent architectures including reasoning systems

(c) SE/FIT/HUT 2002

Standards in Middleware

- Corba
 - Interoperability - objects access each other
 - Services in the environment – Naming service, lookup services etc.
 - POA Interfaces – plug and play middleware interfaces
- Networking standards
 - Routers exchange messages
 - Creates a distributed environment
 - Clients programmed at a level above the TCP/IP stack
- "Hardware" middleware
 - E.g. mobile phone networks...

(c) SE/FIT/HUT 2002

1. Agent Standards

- FIPA
 - Foundation for Intelligent Physical Agents
- Massif
 - OMG effort to standardize mobile agents – middleware services and internal middleware interfaces
- De Facto standards
 - Jini: Sun distributed object layer including directory services, messaging etc. (DARPA GRID is a Jini based agent network)
 - Aglets: IBM mobile agent system.

(c) SE/FIT/HUT 2002

2. FIPA Overview

- Foundation for Intelligent Physical Agents
 - International standards body
 - 60 member companies with heavy involvement from telecommunications companies in particular.
 - First standard in 1997 – “FIPA 97”
 - Since then – FIPA98 and FIPA2000
- Standards in the following main areas
 - Agent communication (language, content language, interaction protocols)
 - Message transport
 - Directory services (DF)
 - Management and naming services (AMS),

(c) SE/FIT/HUT 2002

Who is FIPA?

- FIPA operates through the open international collaboration of member organisations, which are companies and universities active in the agent field.
- Companies: Alcatel, Boeing, British Telecom, Deutsche Telekom, France Telecom, Fujitsu, Hitachi, HP, IBM, Fujitsu, Hewlett Packard, IBM, Intel, Lucent, NEC, NHK, NTT, Nortel, Siemens, SUN, Telia, Toshiba, etc.
- Universities and Research Institutes: GMD, EPFL, Imperial, IRST, etc.
- Government Agencies: DARPA

(c) SE/FIT/HUT 2002

4. FIPA Encodings and package

- Fipa supports three main types of syntax encoding:
 - Extensible Markup Language – XML
 - Proprietary Binary format
 - Lisp-like format
- Separates between Envelopes and Messages

(c) SE/FIT/HUT 2002

5. FIPA Specifications

- For this course:
 - Agent Management
 - FIPA Agent Management XC00023
 - Agent communication
 - FIPA ACL Message Structure XC00061
 - FIPA Communicative Act XC00037
 - FIPA Interaction Protocols XC00025
 - FIPA SL Content Language XC00008
- Specifications and more information
 - <http://www.fipa.org>

(c) SE/FIT/HUT 2002

Thư viện Java Agent Service

Tài liệu tham khảo:

- JAS Specification
- FIPA Abstract Architecture Specification
- Others FIPA Documents Specification (www.fipa.org)

(c) SE/FIT/HUT 2002

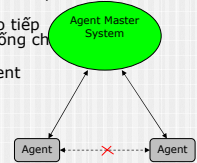
Mục đích

- tổng quan kiến trúc của một hệ đa Agent động do FIPA (Foundation for Intelligent Physical Agent) xây dựng
- Giới thiệu thư viện JAS (Java Agent Service) và một số Use Case để làm rõ flow và kiến trúc của FIPA và JAS và xây dựng một khung agent dựa trên các đặc tả của FIPA và thư viện JAS
- nghiên cứu: các service và cơ chế gửi nhận message trong hệ đa Agent động

(c) SE/FIT/HUT 2002

Hệ đa Agent

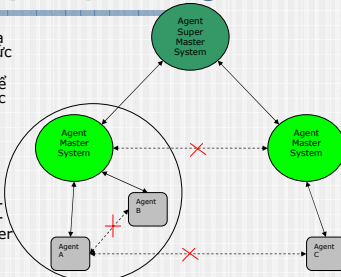
- Hệ thống Agent mà FIPA xây dựng là một hệ đa Agent động: gồm nhiều Agent tương tác với nhau
 - Hệ đa Agent tĩnh: hệ đa Agent có số lượng và chủng loại Agent cố định
 - Hệ đa Agent động: hệ đa Agent có số lượng, chủng loại Agent tham gia hệ thống có thể thay đổi
- Trong hệ đa Agent động, các Agent giao tiếp gián tiếp với nhau thông qua một hệ thống chỉ đóng vai trò môi trường
- Các hành vi cần có trong một hệ đa Agent động:
 - Đăng kí và gỡ bỏ đăng kí
 - Tìm kiếm
 - Gửi và nhận message



(c) SE/FIT/HUT 2002

Mở rộng một hệ đa Agent

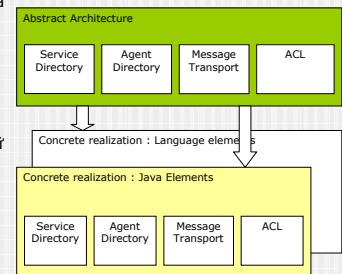
- Đây là mở rộng của một hệ đa Agent động: gồm có nhiều mức Master System
- Mỗi mức Master System có thể coi là một Agent khi xét ở mức Master System cha
- Có duy nhất một mức Master System ở trên cùng
- Các Agent ở cùng một mức không gửi message trực tiếp được cho nhau
- Ví dụ: Mạng máy tính. Server đóng vai trò của Super Master System. PC đóng vai trò Master System



(c) SE/FIT/HUT 2002

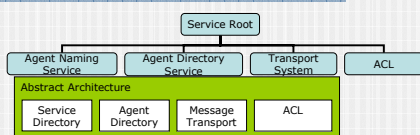
Kiến trúc tổng quan của FIPA về một hệ thống chủ

- Đây là đặc tả trừu tượng của FIPA về một hệ đa Agent động
- 4 thành phần cơ bản của một hệ thống chủ
- 4 thành phần này sẽ được khai báo trong các ngôn ngữ cụ thể như C++, Java, Cobra..



(c) SE/FIT/HUT 2002

Kiến trúc của thư viện JAS



- 5 thành phần cơ bản trong hệ thống JAS
- Tương ứng giữa đặc tả FIPA và JAS
 - Service Root ↔ Service Directory
 - Agent Naming Service & Agent Directory Service ↔ Agent Directory
 - Transport System ↔ Message Transport
 - ACL ↔ ACL

(c) SE/FIT/HUT 2002

Các thao tác chính trong một hệ đa Agent động

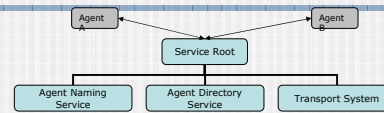
1. Khởi tạo hệ thống agent: khởi tạo các service cần thiết phục vụ cho việc giao tiếp giữa các agent
2. Khởi tạo agent, thiết lập thuộc tính và tham chiếu đến các service
3. Agent giao tiếp với các agent khác và với môi trường
4. Gỡ bỏ đăng kí agent và hủy bỏ hệ thống

(c) SE/FIT/HUT 2002

Các Service của một hệ thống đa Agent động

(c) SE/FIT/HUT 2002

1. Service Directory Service và Service root



- Theo đặc tả FIPA : SDS bản chất là lối vào để Agent truy cập đến các service khác của hệ thống. Điều đó không có nghĩa là Agent bắt buộc phải truy cập đến các Service khác gián tiếp thông qua Service Directory Service mà có thể truy cập trực tiếp
- Theo đặc tả JAS : Service Root là một tập hợp của 3 service là AgentNamingService, AgentDirectoryService và TransportSystem
- Service này được khởi tạo khi hệ thống được khởi tạo, có một instance duy nhất. Instance này được truyền vào làm tham số cho quá trình khởi tạo một Agent

(c) SE/FIT/HUT 2002

2. Agent Naming Service

- Vai trò : cung cấp các phương thức để một Agent nhận về một định danh AgentName trên Master System
- AgentName là một định danh duy nhất trong một hệ thống đa Agent được dùng để phân biệt các Agent với nhau
- AgentName được dùng để đăng kí với hệ thống qua AgentDirectoryService và phục vụ cho việc giao tiếp giữa các Agent
- Trong JAS, AgentName được sinh ra theo một công thức dựa vào địa chỉ của Agent, thời gian yêu cầu tạo AgentName và số lượng Agent tại thời điểm yêu cầu sau đó được mã hoá. Ví dụ AgentName do JAS tạo ra

[agentname: **11d1def534ea1be0:712c4e:f89fdf7166:-7fee**]

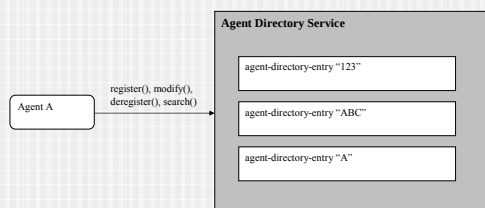
(c) SE/FIT/HUT 2002

3. Agent Directory Service

- Vai trò : chứa các thông tin cần thiết về Agent để phục vụ việc tìm kiếm, thu thập thông tin, trao đổi message giữa các Agent
- Mỗi Agent đăng kí vào Directory Service được lưu trữ thông tin trong một Agent Directory Entry gồm có ít nhất AgentName và AgentLocator

(c) SE/FIT/HUT 2002

Các phương thức mà Service này cung cấp register(), modify(), deregister(), search()



(c) SE/FIT/HUT 2002

Một ví dụ về Agent Directory Entry

```
AgentDescription created ...
class ri.service.directory.BasicAgentDescription
Name: [agentname:
11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
Locators:
[locator:
[properties:
[property:
[fn: javax.agent.locator.port]
[number: 55555]
]
]
[property:
[fn: javax.agent.locator.ip.rmi.id]
[string:
11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
]
[property:
[fn: javax.agent.locator.protocol]
[string: rmi]
]
[property:
[fn: javax.agent.locator.host]
[string: 127.0.0.1]
]
]
]
[agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
[address:
rmi:/127.0.0.1:55555/11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
]
user.attribute3 more.stuff
user.attribute2 fee.fum
user.attribute1 foo.bar
javax.agent.agent-display-name Agent A
AgentDescription registered with AgentDirectoryService.
```

(c) SE/FIT/HUT 2002

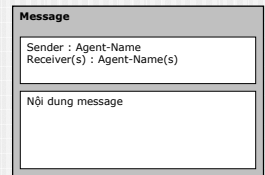
4. Transport System

- Vai trò : cung cấp các phương thức và đối tượng phục vụ việc gửi và nhận message
- 2 thành phần cơ bản của Transport System : Message Structure, Message Transport Service

(c) SE/FIT/HUT 2002

4.1 Message Structure

- Là một cấu trúc gồm có ít nhất 3 thành phần là Sender, Receiver và nội dung message, ngoài ra còn có thể chứa thêm các thuộc tính khác do người dùng định nghĩa
- FIPA đưa ra 1 số thuộc tính chuẩn của message structure
Performative, Sender, Receiver, Reply-to, Content, Language, Encoding, Ontology, Protocol, Conversation-id, Reply-with, Inreply-to, Reply-by

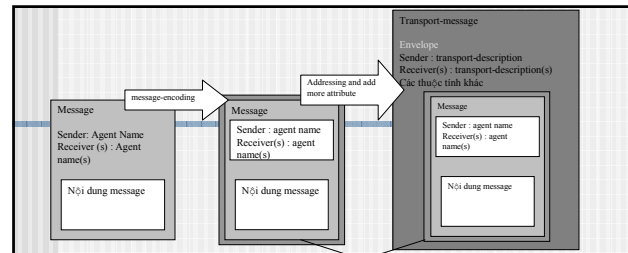


(c) SE/FIT/HUT 2002

Khái niệm Transport-message

- Để message có thể gửi đi sử dụng các giao thức do Message Transport Service cung cấp, message phải được định dạng phù hợp với các giao thức này. Định dạng này gọi là transport-message
- Transport-message gồm 2 thành phần
 - Payload : là kết quả mã hoá message từ dạng object sang dạng có thể truyền đi được. Thông tin chứa trong message và payload là tương đương nhau nhưng ở định dạng khác
 - Envelope : bao gói payload, chứa các thông tin chỉ ra cách thức mã hoá và giao thức truyền nhận transport-message

(c) SE/FIT/HUT 2002



- Chú ý rằng các giá trị Sender và Receiver của Envelope là các transport-description chứa các thông tin về MTS còn các giá trị Sender và Receiver của Payload chứa các thông tin về các Agent như AgentName, chúng ta hãy xem xét ví dụ cụ thể 1 transport message

(c) SE/FIT/HUT 2002

Ví dụ Transport Message

```
Agent A waiting for incoming messages ...
Incoming transport message:
[transportmessage:
  [
    [receiver:
      [locator:
        [properties:
          [property:
            [fqdn: javax.agent.locator.ip.rmi.id]
            [string: 11d1def534ea1be0:712c4e:f89df7166:-7fed]
          ]
          [property:
            [fqdn: javax.agent.locator.host]
            [string: 127.0.0.1]
          ]
          [property:
            [fqdn: javax.agent.locator.port]
            [number: 55555]
          ]
          [property:
            [fqdn: javax.agent.locator.protocol]
            [string: rmi]
          ]
        ]
      ]
    ]
    [type: rmi]
    [address: rmi://127.0.0.1:55555/11d1def534ea1be0:712c4e:f89df7166:-7fed]
  ]
] (còn nữa)
```

(c) SE/FIT/HUT 2002

Ví dụ Transport Message (tiếp)

```
[
  [locator:
    [properties:
      [property:
        [fqdn: javax.agent.locator.host]
        [string: 127.0.0.1]
      ]
      [property:
        [fqdn: javax.agent.locator.ip.rmi.id]
        [string: 11d1def534ea1be0:fabe9:f89df8a7b:-7ff6]
      ]
      [property:
        [fqdn: javax.agent.locator.protocol]
        [string: rmi]
      ]
      [property:
        [fqdn: javax.agent.locator.port]
        [number: 55555]
      ]
    ]
  ]
  [type: rmi]
  [address: rmi://127.0.0.1:55555/11d1def534ea1be0:fabe9:f89df8a7b:-7ff6]
] (bỏ envelope)
```

(c) SE/FIT/HUT 2002

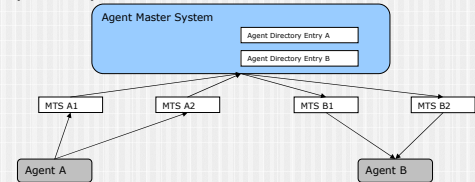
Ví dụ Transport Message (tiếp)

```
[
  [acimessage:
    [performative: org.fipa.acl.inform]
    [sender:
      [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fe2]
    ]
    [receiver:
      [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
    ]
    [content:
      [string: Here I am!]
    ]
  ]
]
```

(c) SE/FIT/HUT 2002

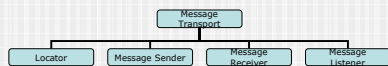
4.2 Message Transport Service

- Vai trò : Cung cấp các phương thức để chuyển đổi giữa message và transport-message, gửi và nhận các transport-message
- Một transport system có thể có nhiều MTS khác nhau. Các Agent có thể sử dụng tùy ý các MTS mà nó được hỗ trợ



(c) SE/FIT/HUT 2002

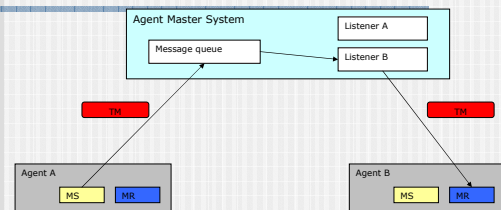
4.3 Cơ cấu truyền nhận transport-message



- Một message transport service gồm 4 thành phần chính
 - Locator : chứa các thông tin về giao thức dùng để truyền tin và địa chỉ của Agent
 - Message Sender : cung cấp các phương thức chuyển message thành transport message và gửi đi
 - Message Receiver : cung cấp các phương thức nhận transport-message và chuyển thành message rồi xử lý
 - Message Listener : thông báo cho message receiver khi có một message được gửi cho Agent

(c) SE/FIT/HUT 2002

Cơ cấu truyền nhận transport-message (tiếp)



- Message Sender của Agent A gửi transport-message tới message queue của Master System, sau đó Master System phân loại và báo cho Message Listener của Agent B biết, Listener này thông báo cho Message Receiver của Agent B để nhận message và xử lý

(c) SE/FIT/HUT 2002

Các Use Case điển hình

- Use Case 0 : khởi tạo hệ thống
 - Use Case 1 : khởi tạo Agent A
 - Use Case 2 : khởi tạo Agent B, Agent B gửi message đến Agent A
 - Use Case 3 : Agent A và Agent B giao tiếp với nhau
- * Phần này sẽ không đi sâu vào phần code của thư viện JAS

(c) SE/FIT/HUT 2002

Use Case 0 : Khởi tạo hệ thống

- Tạo ra một instance của lớp Service Root
- Các lớp và giao diện liên quan: Booter, DefaultBooter, AgentServiceRoot
- Khởi tạo AgentNamingService, AgentDirectoryService, MessageTransportService

(c) SE/FIT/HUT 2002

Kết quả Use Case 0

```

ServiceRoot acquired ...
[ri.service.DefaultServiceRoot@b819f]

AgentNamingService acquired ...
[ri.service.naming.rmi.RMIAgentNamingServiceProxy@6b017e]

MessageTransportService acquired ...
[ri.service.transport.rmi.RMIMessageTransportService@6b13c7]

AgentDirectoryService acquired ...
[ri.service.directory.rmi.RMIAgentDirectoryServiceProxy@2c2f9c]

```

(c) SE/FIT/HUT 2002

Use Case 1 : Agent A khởi tạo, đăng kí và chờ nhận message

- Truyền tham số Service Root vào hàm khởi tạo Agent
- Tham chiếu đến AgentNamingService, thiết lập AgentName
AgentNamingService.getAgentName()
- Tham chiếu đến MessageTransportService
TransportSystem.getMessageTransportService()
 - thiết lập LocalLocator
MessageTransportService.newLocalLocator()
 - thiết lập MessageReceiver
MTS.newMessageReceiver()
 - thiết lập MessageListener
MessageReceiver.addMessageListener() và bind vào
MessageTransportService
- Tham chiếu đến AgentDirectoryService
ServiceRoot.getAgentDirectoryService()
- Agent A tạo AgentDescription và đăng kí với
AgentDirectoryService
- Agent A chờ đợi nhận message

(c) SE/FIT/HUT 2002

Kết quả Use Case 1

```

JAS Use Case I : Agent 'A' starts, registers and waits for messages
=====

ServiceRoot acquired ...
[ri.service.DefaultServiceRoot@b819f]

AgentNamingService acquired ...
[ri.service.naming.rmi.RMIAgentNamingServiceProxy@6b017e]

AgentName acquired from AgentNamingService ...
[agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]

MessageTransportService acquired ...
[ri.service.transport.rmi.RMIMessageTransportService@6b13c7]

```

(c) SE/FIT/HUT 2002

```

AgentLocator bound to MessageTransportService ...
[locator:
  [properties:
    [property:
      [fqcn: javax.agent.locator.port]
      [number: 55555]
    ]
    [property:
      [fqcn: javax.agent.locator.ip.rmi.id]
      [string: 11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
    ]
    [property:
      [fqcn: javax.agent.locator.protocol]
      [string: rmi]
    ]
    [property:
      [fqcn: javax.agent.locator.host]
      [string: 127.0.0.1]
    ]
  ]
  [type: rmi]
  [address: rmi://127.0.0.1:55555/11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
]

```

(c) SE/FIT/HUT 2002

```

AgentDirectoryService acquired ...
[ri.service.directory.rmi.RMIAgentDirectoryServiceProxy@2c2f9c]

AgentDescription created ...
class ri.service.directory.BasicAgentDescription
Name: [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
Locators:
[locator:
  [properties:
    [property:
      [fqcn: javax.agent.locator.port]
      [number: 55555]
    ]
    [property:
      [fqcn: javax.agent.locator.ip.rmi.id]
      [string: 11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
    ]
    [property:
      [fqcn: javax.agent.locator.protocol]
      [string: rmi]
    ]
  ]
]

```

(c) SE/FIT/HUT 2002

```

[property:
  [fqcn: javax.agent.locator.host]
  [string: 127.0.0.1]
]
]
[type: rmi]
[address: rmi://127.0.0.1:55555/11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
]
user.attribute: more.stuff
user.attribute: fee.fum
user.attribute: foo.bar
javax.agent.agent.display-name: Agent A

AgentDescription registered with AgentDirectoryService.
Agent A waiting for incoming messages ...

```

(c) SE/FIT/HUT 2002

Use Case 2 : Agent B khởi tạo và gửi message cho Agent A

- Khởi tạo agent B như Use Case 1
- Agent B tìm kiếm Agent A
 - Xây dựng AgentDescription phù hợp với các thông tin Agent B biết về A ví dụ Agent B tìm kiếm Agent A theo tiêu chuẩn agent-display-name = Agent A
 - Agent B gửi search condition đến AgentDirectoryService
 - ADS trả về một hoặc nhiều agent phù hợp với điều kiện search
- Agent B xây dựng transport-message gửi đến Agent A
 - Thiết lập các lớp AcrElementUtility, AcrElementBuilder, AcrElementHelper, AcrAgentName, AcrAclMessage
 - Thiết lập các thuộc tính : SenderName, ReceiverName, SenderLocator, ReceiverLocator
 - Tạo payload và envelope
- Gửi transport-message

(c) SE/FIT/HUT 2002

Kết quả Use Case 2

JAS Use Case II : Agent 'B' starts and sends message to Agent 'A'

```
=====
AgentName acquired from AgentNamingService ...
[agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fe2]
javax.agent.agent-display-name Agent B
```

<lược bỏ một số đoạn>

AgentDescription registered with AgentDirectoryService.

Agent B ready!

Created search query for Agent 'A'
 class ri.service.directory.BasicAgentDescription
 javax.agent.agent-display-name Agent A

Submitted search query ...

(c) SE/FIT/HUT 2002

```
First matching result :
class ri.service.directory.BasicAgentDescription
Name: [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
Locators:
[
  [
    [properties:
      [property:
        [fqcn: javax.agent.locator.host]
        [string: 127.0.0.1]
      ]
    ]
    [property:
      [fqcn: javax.agent.locator.ip.rmi.id]
      [string: 11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
    ]
    [property:
      [fqcn: javax.agent.locator.protocol]
      [string: rmi]
    ]
    [property:
      [fqcn: javax.agent.locator.port]
      [number: 55555]
    ]
  ]
]
[type: rmi]
[address: rmi://127.0.0.1:55555/11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
}
user.attribute1 foo.bar
user.attribute3 more.stuff
javax.agent.agent-display-name Agent A
user.attribute2 fee.fum
```

(c) SE/FIT/HUT 2002

```
Sending message ...
[transportmessage:
  [
    [
      [locator:
        [properties:
          [property:
            [fqcn: javax.agent.locator.host]
            [string: 127.0.0.1]
          ]
          [property:
            [fqcn: javax.agent.locator.ip.rmi.id]
            [string: 11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
          ]
          [property:
            [fqcn: javax.agent.locator.protocol]
            [string: rmi]
          ]
          [property:
            [fqcn: javax.agent.locator.port]
            [number: 55555]
          ]
        ]
      ]
      [type: rmi]
      [address: rmi://127.0.0.1:55555/11d1def534ea1be0:712c4e:f89fdf7166:-7fed]
    ]
  ]
]
```

(c) SE/FIT/HUT 2002

```
[
  [
    [locator:
      [properties:
        [property:
          [fqcn: javax.agent.locator.port]
          [number: 55555]
        ]
        [property:
          [fqcn: javax.agent.locator.ip.rmi.id]
          [string: 11d1def534ea1be0:fabe9:f89fdf8a7b:-7ff6]
        ]
        [property:
          [fqcn: javax.agent.locator.protocol]
          [string: rmi]
        ]
        [property:
          [fqcn: javax.agent.locator.host]
          [string: 127.0.0.1]
        ]
      ]
    ]
    [type: rmi]
    [address: rmi://127.0.0.1:55555/11d1def534ea1be0:fabe9:f89fdf8a7b:-7ff6]
  ]
]
```

(c) SE/FIT/HUT 2002

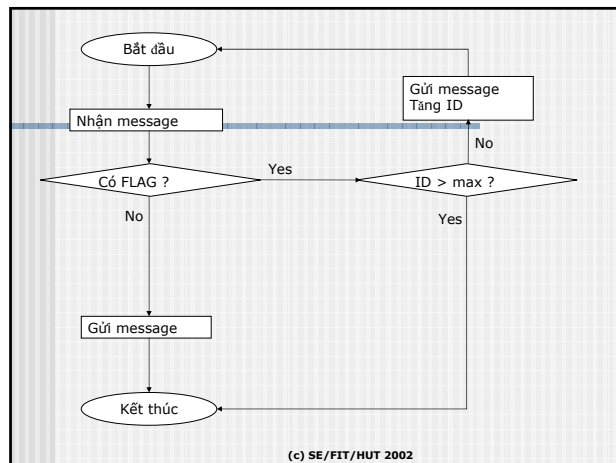
```
[
  [
    [acmessage:
      [performative: org.fipa.aci.inform]
      [sender:
        [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fe2]
      ]
      [receiver:
        [agentname: 11d1def534ea1be0:712c4e:f89fdf7166:-7fee]
      ]
      [content:
        [string: Here I am!]
      ]
    ]
  ]
]
Message sent.
```

(c) SE/FIT/HUT 2002

Use Case 3 : Agent A giao tiếp với Agent B

- Agent A nhận message, xử lý và gửi message trả Agent B
- Quá trình tiếp tục đến khi 1 trong 2 Agent ngừng giao tiếp
- Thuật toán cụ thể sử dụng để xây dựng UC 3
 - Khai báo một tập các message protected static final String[] STATEMENT = { "Good morning !", "Good morning ! How are you today my friend ?", "Oh, I am fine, thank you and you", "So am I ... Let's work !" };
 - Sử dụng một thuộc tính có tên CONVERSATION_FLAG để xác định rằng khi nhận được message, Agent kia có phải trả lời hay không
 - Trong message thêm một thuộc tính conversation-id để xác định trạng thái hiện tại
 - Nội dung của message tương ứng với vị trí của conversation-id trong mảng STATEMENT
 - Ban đầu conversation-id được gán bằng 0, Agent B gửi message cho Agent A
 - Mỗi khi nhận được message, giá trị conversation-id được tăng lên 1
 - Lặp lại việc gửi nhận message đến khi conversation-id lớn hơn kích thước mảng STATEMENT

(c) SE/FIT/HUT 2002



(c) SE/FIT/HUT 2002

Kết quả Use Case 3 (đã lược bớt một số thông tin trùng lặp)

```

Sending message ... (B → A)
[transportmessage:
  [envelope: ]
  [payload: ]
  [aclmessage:
    [properties:
      [property:
        [fq: org.fipa.acl.conversation-id]
        [string: seuss]
      ]
    ]
  ]
  [performative: org.fipa.acl.inform]
  [sender:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fd0]
  ]
  [receiver:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fee]
  ]
  [content:
    [string: Good morning !]
  ]
]
    
```

(c) SE/FIT/HUT 2002

Kết quả UC 3 (tiếp)

```

Incoming transport message: (A → B)
[transportmessage:
  [envelope: ]
  [payload: ]
  [aclmessage:
    [properties:
      [property:
        [fq: org.fipa.acl.conversation-id]
        [string: seuss]
      ]
    ]
  ]
  [performative: org.fipa.acl.inform]
  [sender:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fee]
  ]
  [receiver:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fd0]
  ]
  [content:
    [string: Good morning ! How are you today my friend ?]
  ]
]
    
```

(c) SE/FIT/HUT 2002

Kết quả UC 3 (tiếp)

```

Sending response message ... (B → A)
[transportmessage:
  [envelope: ]
  [payload: ]
  [aclmessage:
    [properties:
      [property:
        [fq: org.fipa.acl.conversation-id]
        [string: seuss]
      ]
    ]
  ]
  [performative: org.fipa.acl.inform]
  [sender:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fd0]
  ]
  [receiver:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fee]
  ]
  [content:
    [string: Oh, I am fine, thanks you and you ]
  ]
]
    
```

(c) SE/FIT/HUT 2002

Kết quả UC 3 (tiếp)

```

Incoming transport message: (A → B)
[transportmessage:
  [envelope: ]
  [payload: ]
  [aclmessage:
    [properties:
      [property:
        [fq: org.fipa.acl.conversation-id]
        [string: seuss]
      ]
    ]
  ]
  [performative: org.fipa.acl.inform]
  [sender:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fee]
  ]
  [receiver:
    [agentname: 11d1def534ea1be0:712c4e:f8b0eba2ec:-7fd0]
  ]
  [content:
    [string: So am I ... Let's work ! ]
  ]
]
Message sent.
    
```

(c) SE/FIT/HUT 2002

JAS Installation Guide

- Cài đặt thư viện JAS
- Chạy các Use Case
- → Xem tài liệu document đi kèm

(c) SE/FIT/HUT 2002

Một số câu hỏi và các vấn đề chưa giải quyết được

(c) SE/FIT/HUT 2002

Câu hỏi 1

- Tại sao lại cần phải quản lý tập trung, tại sao cần có Service Root và định danh Agent ?

(c) SE/FIT/HUT 2002

Câu hỏi 2

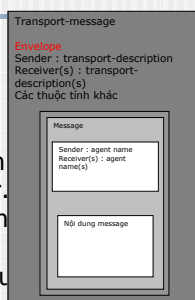
- Trong một mạng máy tính với các giao thức không hỗ trợ các đối tượng được truyền qua mạng. Vậy tham số Service Root là đối tượng để khởi tạo Agent được lấy ở đâu ?

(c) SE/FIT/HUT 2002

Câu hỏi 3

- Một transport-message gồm có 2 thành phần là payload và envelope. Trong envelope đã xác định transport-description của cả sender và receiver. Như vậy MTS được sử dụng để gửi và nhận dữ liệu đã được xác định ngay từ đầu. Phải chăng điều đó là không linh hoạt ?

(c) SE/FIT/HUT 2002



Câu hỏi 4

- Một Agent Directory Service hỗ trợ các phương thức register, modify, deregister, search. Mỗi lời gọi đến phương thức này sẽ gửi các thông tin, yêu cầu đến Master System tức là phải thông qua Message Transport Service nào đó, tuy nhiên khi khởi tạo, agent có thể nhận về AgentName trước khi tham chiếu đến MTS

```
ServiceRoot acquired ...
[In.service.DefaultServiceRoot@6819f]

AgentNamingService acquired ...
[In.service.naming.rmi.RMIAgentNamingServiceProxy@6b017e]

AgentName acquired from AgentNamingService ...
[agentname: 31d1de524aa1be0:712c4e:889d7166-7fee]

MessageTransportService acquired ...
[In.service.transport.rmi.RMIMessageTransportService@6b13c7]
```

- Tại sao lại thế ?

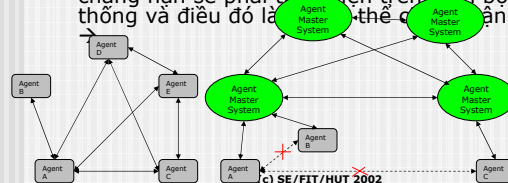
(c) SE/FIT/HUT 2002

Cảm ơn !

(c) SE/FIT/HUT 2002

1. Tại sao cần quản lý tập trung, Service Root và định danh Agent

- Đây là một hệ đa Agent động, số lượng, kiểu Agent có thể thay đổi. Quản lý tập trung, Service Root sẽ giúp việc quản lý Agent dễ dàng hơn, tăng khả năng đáp ứng của hệ thống
- Hãy tưởng tượng 1 hệ thống không có Service Root sẽ như thế này : việc tìm kiếm 1 Agent chẳng hạn sẽ phải thực hiện trên toàn bộ hệ thống và điều đó là không thể chấp nhận được

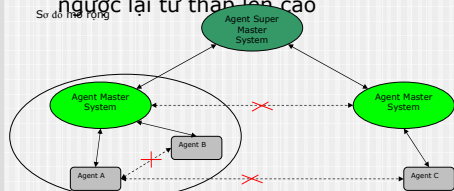


(c) SE/FIT/HUT 2002

Câu 1 (tiếp)

- Do đó định danh của 1 Agent cần được quản lý tập trung
- Trong sơ đồ mở rộng : khi một Agent đăng kí, nó cần được đăng kí ở mức hệ thống từ cao xuống thấp còn khi gỡ bỏ đăng kí thì theo chiều ngược lại từ thấp lên cao

Sơ đồ mở rộng

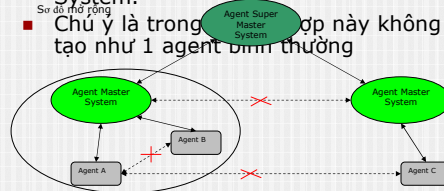


(c) SE/FIT/HUT 2002

2. Service Root được lấy ở đâu

- Trước hết cần phải xem lại sơ đồ hệ đa Agent mở rộng
- Trong câu hỏi này, 1 thành phần đã bị ẩn đi đó là PC, mỗi PC, hệ điều hành là một Sub-Master System.
- Chú ý là trong sơ đồ này không được khởi tạo như 1 agent bình thường

Sơ đồ mở rộng



(c) SE/FIT/HUT 2002

Open Agent Architecture

(c) SE/FIT/HUT 2002

Kiến trúc agent mở (OAA)

- Tổng quan
- Các khái niệm cơ bản
- Các thành phần chính trong OAA
- Lợi ích sử dụng OAA
- So sánh OAA với công nghệ phân tán

(c) SE/FIT/HUT 2002

Các khái niệm cơ bản trong OAA

■ Định nghĩa OAA :

- OAA (Kiến trúc Agent mở) là một framework trong đó một cộng đồng các agent phần mềm chạy trên các máy phân tán có thể làm việc với nhau theo các nhiệm vụ được con người hoặc không phải con người tham gia trong cộng đồng ủy quyền.
- Sự đồng điều hành phân tán và truyền thông mức cao là hai ý tưởng trung tâm đối với nền tảng của OAA.

(c) SE/FIT/HUT 2002

Tổng quan

■ OAA :

- Là một Framework tích hợp việc truyền thông giữa các Software Agent trong môi trường phân tán, thông qua ủy quyền, điều phối...
- Mềm dẻo, thích nghi được với sự tương tác giữa các công cụ phân tán thông qua ủy quyền các tác vụ, các yêu cầu dữ liệu và tự kích hoạt (trigger).

(c) SE/FIT/HUT 2002

Các đặc tính của OAA

Tính mở:

Các agent có thể được tạo ra từ nhiều ngôn ngữ và giao diện trong các hệ thống đã có.

Tính mở rộng:

Các agent có thể được thêm hoặc thay thế một cách linh hoạt

Tính phân tán:

Agent được truyền qua rất nhiều máy tính

Tính song song:

Có thể thực hiện song song các tác vụ con

Kiến trúc mức cao (high level):

An các phần mềm và phần cứng phụ thuộc

Tính đa hình:

Có thể kết hợp cùng nhau các cách thức: viết, nói, điều bộ, vận động.

(c) SE/FIT/HUT 2002

Các khái niệm cơ bản

■ Agent trong OAA:

- Trong OAA, một agent được định nghĩa như một phần mềm tuân theo các chuẩn chức năng và truyền thông đưa ra bởi OAA
- Các agent ủy quyền và nhận nhiệm vụ thông qua một agent điều phối là Facilitator
- Các yêu cầu được diễn tả trong một ngôn ngữ chung cho mọi agent - ICL (Interagent Communication Language)

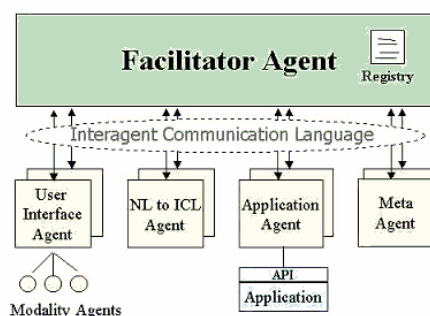
(c) SE/FIT/HUT 2002

Tiến trình điều khiển trong OAA

- *requester* có thể chọn để chỉ ra một cách từng phần làm thế nào để một mục đích được thực hiện bởi một cộng đồng trong một ngôn ngữ giàu tính trực quan.
- *Service providers*, trong suốt quá trình đăng ký với một agent Facilitator, có thể chỉ ra mã điều kiện kiểm tra và ước lượng hiệu năng nó sẽ cho biết khi nào và dưới hoàn cảnh nào thì agent tiếp nhận công việc.
- *Facilitator agent* sử dụng các chiến lược chọn đường độc lập miễn đối với việc chia công việc cho cộng đồng các agent dựa trên một yêu cầu đến và các mô tả khả năng cung cấp dịch vụ.
- Tập hợp các *meta-agent* (các chuyên gia miền) có thể cộng tác để cung cấp Facilitator với các tri thức xác định miền mà hướng dẫn làm thế nào các nhiệm vụ được thực hiện bởi toàn cộng đồng.

(c) SE/FIT/HUT 2002

Các thành phần trong OAA



(c) SE/FIT/HUT 2002

Các thành phần chính

- Facilitator agent
- Meta agent
- Application agent
- User Interface agent
- NL to ICL agent
- Trigger
- Giao thức
- Các agent hỗ trợ khác

(c) SE/FIT/HUT 2002

Facilitator agent

- Facilitator là một agent đặc biệt cung cấp khả năng phối hợp các agent
- Một hệ thống có thể có một hay nhiều Facilitator agent
- Các Facilitator agent có thể phối hợp quản lí, trao đổi với nhau

(c) SE/FIT/HUT 2002

Facilitator agent

3 chức năng chính của Facilitator

1. Duy trì một đăng ký của dịch vụ agent và các khai báo dữ liệu lên blackboard.
2. Lập kế hoạch một chiến lược thực hiện cho một câu truy vấn.
3. Thực hiện một kế hoạch: phân chia các nhiệm vụ thành các nhiệm vụ nhỏ hơn, liên kết các dịch vụ nhỏ với các nhà cung cấp dịch vụ, chọn đường và thu thập thông tin trong số các thành phần tham dự phân tán

(c) SE/FIT/HUT 2002

Facilitator agent

Vấn đề :

Trong các mô hình đơn giản, tại đó chỉ có một Facilitator phục vụ một số các agent client, thì Facilitator có thể bị tắc nghẽn trong truyền thông và một điểm đơn bị lỗi đối với toàn hệ thống.

(c) SE/FIT/HUT 2002

Application agent

- Cung cấp một tập các dịch vụ của một nhu cầu cụ thể. Các ứng dụng này có thể là các công nghệ độc lập miền

Ví dụ: nhận dạng tiếng nói, xử lý ngôn ngữ tự nhiên, email, data retrieval, data mining ...vv

Hoặc user-specific hay domain-specific

Ví dụ: agent lập kế hoạch du lịch.

Các ứng dụng agent có thể dựa trên các thư viện trong đó agent có thể được tạo thông qua các lời gọi hàm API.

(c) SE/FIT/HUT 2002

Meta agent

- Giúp đỡ Facilitator trong việc phối hợp hoạt động của các agent khác.
- Trong khi facilitator dùng phối hợp độc lập miền, meta-agent sử dụng miền và các tri thức xác định ứng dụng hoặc các rules, thuật toán học, lập kế hoạch.

(c) SE/FIT/HUT 2002

User Interface agent

- Có thể bao gồm một tập các agent
- Mỗi agent giám sát một kiểu đầu vào khác nhau và cộng tác để đưa ra một cách diễn giải tốt nhất cho các đầu vào hiện tại.

(c) SE/FIT/HUT 2002

Client agent

- Các Agent trong hệ thống OAA là một Facilitator Agent hoặc là một Client Agent
- Khi được gọi đến, một Client Agent tạo một kết nối tới một Facilitator, được biết như là Parent Facilitator của nó, và báo cho Facilitator các dịch vụ nó cung cấp
- Các loại dịch vụ: đáp ứng các mục đích theo thủ tục, duy trì và truy vấn các kho lưu trữ dữ liệu, thiết lập và trả lời tới bộ kích hoạt của các loại chung

(c) SE/FIT/HUT 2002

Ngôn ngữ truyền thông ICL

- Tất cả các truyền thông liên agent đều sử dụng một ngôn ngữ xác định trong OAA, đó là ICL (Interagent Communication Language)
- Đặc điểm: phân tách câu hỏi *What* từ câu hỏi *How*. Các tham số điều khiển chỉ ra làm thế nào các nhiệm vụ có thể thực hiện, ai nên thực hiện nhiệm vụ, và điều phân tách từ các mô tả bản thân nhiệm vụ.

(c) SE/FIT/HUT 2002

Các thành phần trong ICL

- Khai báo các khả năng (solvable)
- Các sự kiện
- Các yêu cầu dịch vụ
- Các phần tử dữ liệu
- Trigger
- Khai báo tham số
- Tên và địa chỉ agent

(c) SE/FIT/HUT 2002

Trigger

- OAA cung cấp một cơ chế giúp Agent có thể định rõ một hành động khi tập hợp các điều kiện của chúng được thỏa mãn. Mỗi Agent có thể cài đặt các triggers một cách cục bộ (trong chính Agent đó) hoặc từ xa (trong Facilitator của nó) hoặc tại một Agent ngang hàng.
- Có 4 loại trigger: task, com, data, và time

(c) SE/FIT/HUT 2002

Trigger

- Các loại trigger:
 - Task*: theo dõi các điều kiện gắn với agent, ví dụ như “Khi email đến, thông báo cho tôi.”
 - Com(munication)*: tìm kiếm các sự kiện truyền thông agent mức thấp, ví dụ như “Khi nào một sự kiện agent ngắt kết nối, hãy ghi lại thời gian.”
 - Data*: nắm bắt sự thay đổi của thông tin, ví dụ “Khi tình trạng điện thoại rảnh, hãy báo cho tôi.”
 - Time*: các sự kiện hướng thời gian, ví dụ như “Ngày thứ 5, lúc 14h gửi cho tôi một phiếu mời họp mặt.”

(c) SE/FIT/HUT 2002

Các agent hỗ trợ trong OAA

- Agent phân tích ngôn ngữ tự nhiên.
- Agent nhận dạng tiếng nói.
- Agent text-to-speech.
- Agent cơ sở dữ liệu.
- Agent điện thoại (cho phép mobile truy nhập vào nhận dạng tiếng nói từ các vị trí từ xa).
-vv (monitor, debug)

(c) SE/FIT/HUT 2002

Lợi ích sử dụng OAA

- Cung cấp các kỹ thuật đa năng trong liên kết (interoperation) và phối hợp hoạt động.
- Giao diện sử dụng gần gũi với con người
- Các yêu cầu công nghệ phần mềm (tái sử dụng, ứng dụng linh hoạt ..vvv)
- Kết hợp các hệ thống thương mại với các công nghệ trí tuệ nhân tạo, ví dụ như lập kế hoạch, nhận dạng tiếng nói, ngôn ngữ tự nhiên,vv...

(c) SE/FIT/HUT 2002

Hướng phát triển

- Nhiều công nghệ được tích hợp vào OAA. Tăng số lượng các hệ thống được xây dựng để trình diễn khả năng của các công nghệ này.
- Tuy nhiên, sự phát triển trong công nghệ sẽ tiếp tục trong nhiều năm nữa và hy vọng OAA sẽ tiếp tục là một phần của sự phát triển.

(c) SE/FIT/HUT 2002